

Proyecto 2:

Clasificación de comandos de voz para la selección automática del modo de operación del robot Husky A200

Marlon Gutiérrez Vásquez, C33619

Escuela de Ingeniería Eléctrica

Universidad de Costa Rica

IE0435 - Inteligencia Artificial Aplicada a la Ingeniería Eléctrica, I-2026, Grupo 01

Profesor: Marvin Coto Jiménez.

Resumen—En este proyecto se desarrolló un sistema de reconocimiento e interpretación de comandos de voz mediante técnicas de aprendizaje automático, con el propósito de automatizar la selección del modo de operación de un robot móvil Husky A200 simulado en Gazebo bajo el sistema operativo Ubuntu 20.04.6 LTS utilizando ROS Noetic.

El robot ya contaba con un nodo de ROS capaz de ejecutar cuatro modos de operación diferentes: desplazamiento hacia un objetivo específico, seguimiento de una trayectoria parcial, recorrido completo de todos los objetivos registrados y recorrido cíclico. Sin embargo, la selección de estos modos requería que el usuario ingresara manualmente el modo de operación, el objetivo inicial y el objetivo final mediante un archivo de parámetros.

El objetivo principal de este proyecto consistió en eliminar dicha intervención manual, permitiendo que estos parámetros fueran obtenidos automáticamente a partir de un comando de voz emitido por el usuario. Para ello se desarrolló un sistema compuesto por dos programas principales. El primero se encarga del entrenamiento y evaluación de distintos modelos de clasificación supervisada utilizando un conjunto de datos conformado por comandos de voz previamente etiquetados. El segundo corresponde a un nodo de ROS encargado de capturar el audio mediante un micrófono, convertir el habla a texto utilizando el motor de reconocimiento de voz Vosk, interpretar el comando mediante el modelo previamente entrenado y generar automáticamente el archivo de parámetros requerido por el robot.

Para el entrenamiento se utilizaron 607 comandos de voz etiquetados, mientras que para la etapa de prueba se emplearon 153 comandos independientes. Los modelos evaluados fueron Árbol de Decisión, Naive Bayes, K-Nearest Neighbors, Máquina de Soporte Vectorial lineal y Máquina de Soporte Vectorial con kernel RBF. La comparación se realizó utilizando las métricas Accuracy, Precision, Recall y F1-score.

A partir de los resultados obtenidos, el modelo SVM con kernel RBF presentó el mejor desempeño general, alcanzando un F1-score ponderado de 0.9674. Por esta razón, fue seleccionado como el modelo definitivo para su integración con el nodo de reconocimiento de voz. Los resultados obtenidos demuestran que el sistema desarrollado permite automatizar satisfactoriamente la selección del modo de operación del robot Husky A200 mediante comandos de voz.

Debido al alcance del presente informe, el énfasis se centra en la descripción general de la implementación del sistema de clasificación de comandos de voz y en el análisis de los resultados

obtenidos. Por esta razón, no se desarrolla una explicación detallada de herramientas y sistemas auxiliares, como ROS Noetic, Vosk y la arquitectura completa del sistema de navegación, ya que ello excedería el objetivo principal del proyecto. Además, parte de la infraestructura utilizada corresponde a desarrollos realizados en un proyecto previo, cuyo contenido no es permitido ser descrito en detalle.

Palabras clave—Clasificación supervisada, comandos de voz, SVM RBF, ROS, Vosk, Husky A200, TF-IDF, Accuracy, Precision, Recall, F1-score.

I. OBJETIVOS

I-A. *Objetivo general:*

Desarrollar e implementar un sistema basado en aprendizaje automático capaz de interpretar comandos de voz para automatizar la selección del modo de operación del robot móvil Husky A200.

I-B. *Objetivos específicos:*

- Construir un conjunto de datos conformado por comandos de voz etiquetados para el entrenamiento.
- Comparar diferentes algoritmos de aprendizaje automático utilizando métricas de desempeño como Accuracy, Precision, Recall y F1-score, con el fin de seleccionar el modelo más adecuado para el problema planteado.
- Integrar el modelo de clasificación seleccionado con un nodo de ROS que emplee el motor de reconocimiento de voz Vosk para interpretar comandos emitidos por el usuario.
- Validar experimentalmente el funcionamiento del sistema completo mediante pruebas realizadas con comandos de voz reales, para verificar la correcta identificación de la intención del usuario.

II. INTRODUCCIÓN

En los últimos años, el empleo de técnicas de inteligencia artificial ha permitido desarrollar sistemas capaces de interpretar información proveniente del lenguaje natural para facilitar la

interacción entre las personas y diferentes dispositivos tecnológicos. En particular, el aprendizaje automático supervisado ha demostrado ser una herramienta eficaz para resolver problemas de clasificación de texto, permitiendo identificar automáticamente la intención de un usuario a partir de información previamente etiquetada [1], [2].

En aplicaciones robóticas, la incorporación de interfaces basadas en comandos de voz representa una alternativa natural para mejorar la interacción entre el operador y el sistema. Mediante el uso de motores de reconocimiento de voz es posible transformar la señal de audio en texto, el cual posteriormente puede ser procesado por algoritmos de aprendizaje automático para interpretar la acción solicitada por el usuario [8], [9].

El presente proyecto se desarrolló sobre un robot móvil Husky A200 simulado en Gazebo utilizando ROS Noetic. El sistema de navegación empleado ya disponía de diferentes modos de operación implementados previamente; sin embargo, la selección del modo de operación y de los objetivos de navegación debía realizarse mediante la modificación manual de un archivo de parámetros. Con el propósito de automatizar este procedimiento, se desarrolló un sistema capaz de interpretar comandos de voz, identificar automáticamente la intención del usuario y generar los parámetros requeridos por el sistema de navegación.

Para ello se construyó un conjunto de datos compuesto por comandos de voz etiquetados, el cual fue utilizado para entrenar y comparar diferentes algoritmos de clasificación supervisada. Los modelos evaluados incluyen Árbol de Decisión, Naive Bayes, K-Nearest Neighbors (KNN), Máquinas de Soporte Vectorial (SVM) lineales y SVM con kernel RBF, seleccionando finalmente el clasificador con mejor desempeño de acuerdo con las métricas Accuracy, Precision, Recall y F1-score.

Debido al alcance del presente informe, el énfasis se centra en la descripción general de la implementación del sistema de clasificación y en el análisis de los resultados obtenidos. Por esta razón, no se desarrolla una explicación detallada de todas las herramientas empleadas durante la implementación, como ROS Noetic, Gazebo o el motor de reconocimiento de voz Vosk, ya que ello excedería el objetivo principal del proyecto. Asimismo, parte de la infraestructura utilizada corresponde a un sistema de navegación desarrollado previamente, cuyo funcionamiento interno no forma parte del alcance de este documento.

III. ESTRUCTURA GENERAL DEL PROYECTO

La organización final del proyecto fue preparada para facilitar el entrenamiento del modelo de clasificación, la integración con ROS y el almacenamiento de todos los archivos necesarios para el funcionamiento del sistema. Para ello se utilizó un directorio principal denominado `voice_control`, dentro del cual se agrupan los conjuntos de datos, el modelo entrenado y los programas desarrollados.

Los principales directorios utilizados fueron los siguientes:

- **data:** contiene los archivos `train_commands.csv` y `test_commands.csv`, utilizados durante el entrenamiento y la evaluación de los modelos de clasificación.
- **models:** almacena el archivo `mejor_modelo.joblib`, correspondiente al modelo definitivo seleccionado después del proceso de evaluación.
- **src:** contiene los programas principales desarrollados en Python. El archivo `train_voice_command_models.py` implementa el entrenamiento y evaluación de los modelos, mientras que `voice_command_node.py` implementa el nodo de ROS encargado de interpretar comandos de voz en tiempo real.

La Fig. 1 muestra la organización general del proyecto desarrollado.

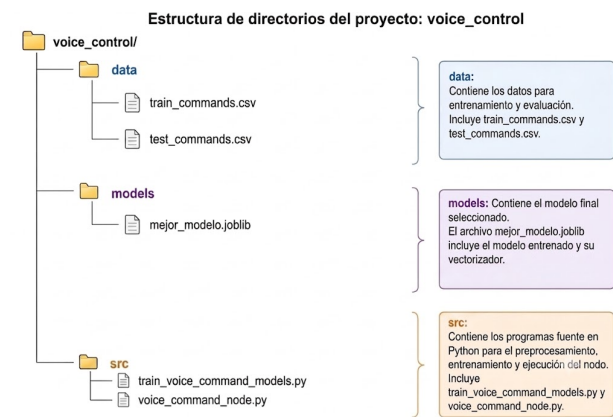


Figura 1. Estructura general del proyecto.

IV. METODOLOGÍA PROPUESTA

El objetivo principal de este proyecto consistió en desarrollar un sistema capaz de interpretar comandos de voz para automatizar la selección del modo de operación del robot móvil Husky A200. Para ello fue necesario diseñar una metodología que permitiera transformar instrucciones emitidas en lenguaje natural en información estructurada que posteriormente pudiera ser utilizada por un algoritmo de clasificación supervisada.

La metodología desarrollada comprende tres etapas principales. En primer lugar, se construyó un conjunto de datos conformado por comandos de voz previamente etiquetados. Posteriormente, los comandos fueron transformados en una representación numérica adecuada para el entrenamiento mediante la técnica TF-IDF. Finalmente, esta información fue utilizada para entrenar y evaluar distintos algoritmos de clasificación supervisada con el propósito de seleccionar el modelo con mejor desempeño para su integración con el sistema de reconocimiento de voz.

IV-A. Planteamiento del problema

El sistema de navegación utilizado en este proyecto ya disponía de cuatro modos de operación previamente implementados, correspondientes al desplazamiento hacia un

objetivo específico, recorrido parcial, recorrido completo y recorrido cíclico. Sin embargo, la selección del modo de operación y de los objetivos asociados debía realizarse manualmente mediante la modificación del archivo `operation_mode_parameters.txt`, lo que hacía que la interacción con el sistema resultara poco práctica para el usuario.

Con el propósito de automatizar este procedimiento, se planteó el desarrollo de un sistema basado en aprendizaje automático capaz de interpretar comandos de voz emitidos en lenguaje natural e identificar automáticamente la intención correspondiente. A partir de dicha predicción, el sistema genera los parámetros requeridos por el algoritmo de navegación, eliminando la necesidad de modificar manualmente los archivos de configuración.

IV-B. Construcción del conjunto de datos

Para entrenar un modelo de clasificación supervisada fue necesario construir un conjunto de datos compuesto por comandos de voz previamente etiquetados. Cada muestra representa una instrucción que un usuario podría pronunciar para controlar el robot durante la navegación.

Debido a que una misma acción puede expresarse utilizando diferentes palabras o estructuras gramaticales, se incorporaron múltiples variaciones del lenguaje cotidiano. Esto permite que el modelo aprenda patrones generales del lenguaje en lugar de memorizar únicamente frases específicas.

Cada comando almacenado dentro del conjunto de datos contiene cuatro atributos principales:

- Texto completo del comando.
- Intención correspondiente al comando.
- Objetivo inicial.
- Objetivo final.

La intención constituye la variable objetivo del problema de clasificación y corresponde directamente a uno de los cuatro modos de operación implementados por el sistema de navegación.

IV-C. Preparación de las características

Los algoritmos de aprendizaje automático no pueden procesar directamente información textual, por lo que fue necesario transformar cada comando de voz en una representación numérica. Para ello se empleó la técnica *Term Frequency-Inverse Document Frequency* (TF-IDF), ampliamente utilizada en problemas de clasificación de texto [2].

TF-IDF asigna un peso a cada palabra de acuerdo con su frecuencia de aparición dentro de un comando y con la importancia que dicha palabra tiene respecto al conjunto completo de documentos. De esta manera, términos frecuentes dentro de un comando, pero poco comunes en el resto del conjunto de datos, adquieren un mayor peso durante el entrenamiento del modelo.

La representación obtenida mediante TF-IDF fue utilizada como entrada para todos los algoritmos de clasificación evaluados, garantizando que la comparación entre modelos se realizara bajo las mismas condiciones experimentales.

IV-D. Proceso de entrenamiento

Una vez construidos los conjuntos de datos y obtenida la representación numérica de los comandos mediante TF-IDF, se procedió al entrenamiento de diferentes modelos de clasificación supervisada. Todos los algoritmos fueron entrenados utilizando exactamente el mismo conjunto de entrenamiento, con el propósito de garantizar una comparación objetiva entre los distintos enfoques evaluados.

Durante esta etapa se implementó un programa denominado `train_voice_command_models.py`, encargado de automatizar completamente el proceso de entrenamiento. Inicialmente, el programa carga los conjuntos de entrenamiento y prueba, posteriormente aplica la transformación TF-IDF a los comandos de voz y utiliza las características obtenidas para entrenar cada uno de los modelos considerados.

Los algoritmos implementados corresponden a Árbol de Decisión, Naive Bayes, K-Nearest Neighbors (KNN), Máquina de Soporte Vectorial (SVM) lineal y Máquina de Soporte Vectorial con kernel RBF. Una vez finalizado el entrenamiento, cada modelo es evaluado utilizando el mismo conjunto de prueba mediante las métricas Accuracy, Precision, Recall y F1-score.

Finalmente, el programa compara el desempeño obtenido por cada clasificador y selecciona automáticamente el modelo con mayor F1-score ponderado. El modelo seleccionado se almacena en el archivo `mejor_modelo.joblib`, el cual incluye tanto el clasificador entrenado como el vectorizador TF-IDF empleado durante el proceso de aprendizaje. Este archivo constituye el modelo definitivo utilizado posteriormente por el nodo de ROS para realizar nuevas predicciones sin necesidad de repetir el entrenamiento.

V. EVALUACIÓN DE LOS MODELOS DE CLASIFICACIÓN

Una vez finalizada la etapa de entrenamiento, se procedió a evaluar objetivamente el desempeño de cada uno de los clasificadores implementados. Para ello se utilizó el conjunto de prueba compuesto por 153 comandos de voz independientes, el cual no fue empleado durante el entrenamiento. Esto permitió medir la capacidad de generalización de cada modelo frente a ejemplos nuevos, evitando que la evaluación se realizara sobre los mismos datos utilizados durante el aprendizaje.

Todos los algoritmos fueron evaluados bajo exactamente las mismas condiciones experimentales, utilizando el mismo conjunto de entrenamiento, la misma representación mediante TF-IDF y el mismo conjunto de prueba. De esta forma, las diferencias observadas en el desempeño corresponden únicamente a las características propias de cada algoritmo de clasificación.

V-A. Modelos de clasificación evaluados

Con el propósito de determinar cuál algoritmo resultaba más adecuado para interpretar comandos de voz, se implementaron cinco modelos de clasificación supervisada ampliamente utilizados en problemas de procesamiento de texto.

Los modelos evaluados fueron los siguientes:

- **Árbol de Decisión:** modelo basado en una estructura jerárquica de decisiones sucesivas, donde cada nodo evalúa una característica hasta determinar la clase correspondiente [3].
- **Naive Bayes:** clasificador probabilístico fundamentado en el teorema de Bayes, el cual supone independencia estadística entre las características utilizadas durante el entrenamiento [4].
- **K-Nearest Neighbors (KNN):** algoritmo que asigna la clase de una nueva muestra considerando las clases predominantes entre sus vecinos más cercanos dentro del espacio de características [5].
- **Máquina de Soporte Vectorial (SVM) lineal:** clasificador que busca separar las diferentes clases mediante un hiperplano lineal que maximiza la distancia entre ellas [6].
- **Máquina de Soporte Vectorial con kernel RBF:** variante de SVM que emplea un kernel de base radial para construir fronteras de decisión no lineales, permitiendo modelar relaciones más complejas entre las características del problema [6].

V-B. Métricas de evaluación

El desempeño de los diferentes modelos fue evaluado mediante las métricas Accuracy, Precision, Recall y F1-score, las cuales constituyen indicadores ampliamente utilizados en problemas de clasificación supervisada [7].

La métrica *Accuracy* representa la proporción de comandos correctamente clasificados respecto al número total de muestras evaluadas.

La *Precision* indica qué proporción de las predicciones realizadas por el modelo corresponde realmente a la clase correcta.

El *Recall* mide la capacidad del clasificador para identificar correctamente las muestras pertenecientes a cada una de las clases presentes en el conjunto de datos.

Finalmente, el *F1-score* corresponde a la media armónica entre Precision y Recall, proporcionando una medida equilibrada del desempeño del clasificador cuando ambas métricas tienen igual importancia. Debido a ello, el F1-score ponderado fue utilizado como criterio principal para seleccionar el modelo definitivo.

V-C. Resultados obtenidos

Una vez completado el entrenamiento de todos los modelos, el programa generó automáticamente las métricas correspondientes a cada clasificador utilizando el conjunto de prueba. Los resultados obtenidos se presentan en la Tabla I.

V-D. Selección del modelo definitivo

Considerando los resultados obtenidos, se seleccionó el modelo SVM con kernel RBF como clasificador definitivo del sistema. La elección se fundamentó en que este algoritmo alcanzó el mayor valor de F1-score ponderado, además de obtener simultáneamente los mejores valores de Accuracy, Precision y Recall entre todos los modelos evaluados.

Tabla I
COMPARACIÓN DEL DESEMPEÑO DE LOS MODELOS EVALUADOS.

Modelo	Accuracy	Precision	Recall	F1-score
SVM RBF	0.9673	0.9677	0.9673	0.9674
SVM Lineal	0.9608	0.9619	0.9608	0.9608
Naive Bayes	0.9477	0.9464	0.9477	0.9463
Árbol de Decisión	0.9346	0.9421	0.9346	0.9340
KNN ($k = 5$)	0.9216	0.9248	0.9216	0.9214

Una vez seleccionado el clasificador, el programa almacenó automáticamente el modelo entrenado en el archivo `mejor_modelo.joblib`. Este archivo contiene tanto el modelo de clasificación como el vectorizador TF-IDF utilizado durante el entrenamiento, permitiendo que posteriormente el nodo de ROS pueda realizar nuevas predicciones sin necesidad de repetir el proceso de aprendizaje.

La disponibilidad del modelo previamente entrenado reduce significativamente el tiempo de ejecución del sistema y facilita su integración con aplicaciones en tiempo real, como el reconocimiento de comandos de voz implementado en este proyecto.

VI. INTEGRACIÓN DEL MODELO Y VALIDACIÓN EXPERIMENTAL

Una vez seleccionado el modelo con mejor desempeño, se procedió a integrarlo dentro del nodo de ROS encargado de interpretar comandos de voz en tiempo real. El objetivo de esta etapa consistió en validar el funcionamiento completo del sistema desarrollado, verificando que fuera capaz de reconocer un comando pronunciado por el usuario, interpretar correctamente su intención y generar automáticamente los parámetros requeridos por el sistema de navegación del robot Husky A200.

La integración se realizó utilizando el archivo `mejor_modelo.joblib`, generado durante la etapa de entrenamiento. Dicho archivo contiene tanto el clasificador entrenado como el vectorizador TF-IDF empleado para transformar los comandos de voz en la representación numérica utilizada durante el aprendizaje. Gracias a ello, el nodo puede realizar nuevas predicciones sin necesidad de repetir el proceso de entrenamiento cada vez que inicia su ejecución.

VI-A. Funcionamiento del nodo de reconocimiento de voz

El sistema desarrollado se implementó mediante un nodo de ROS denominado `voice_command_node.py`, el cual permanece ejecutándose continuamente mientras espera la emisión de nuevos comandos por parte del usuario.

Inicialmente, el nodo adquiere la señal de audio utilizando el micrófono conectado al computador. Posteriormente, el motor de reconocimiento de voz Vosk convierte continuamente dicha señal en una representación textual, permitiendo que el sistema procese comandos emitidos en lenguaje natural [9].

Con el propósito de evitar activaciones accidentales, el algoritmo permanece en estado de espera hasta detectar la

palabra de activación “Husky”. Una vez identificada esta palabra, el nodo comienza a almacenar el contenido del comando pronunciado por el usuario.

La captura del comando continúa hasta detectar cualquiera de las palabras de finalización definidas para el sistema, correspondientes a “ejecuta” o “ejecute”. En ese momento, el nodo considera que el comando ha concluido y procede a procesar la información obtenida.

Posteriormente, el texto reconocido es sometido a un proceso de limpieza que elimina la palabra de activación, las palabras utilizadas para finalizar la grabación y otros términos que no aportan información relevante para el proceso de clasificación.

A continuación, mediante expresiones regulares se identifican automáticamente los objetivos mencionados dentro del comando, permitiendo extraer el objetivo inicial y el objetivo final cuando estos se encuentran presentes en la instrucción.

Finalmente, el comando limpio es transformado utilizando el mismo vectorizador TF-IDF empleado durante el entrenamiento y suministrado al modelo almacenado en el archivo `mejor_modelo.joblib`. A partir de la predicción obtenida, el nodo genera automáticamente el archivo `operation_mode_parameters.txt`, el cual contiene el método de operación, el objetivo inicial y el objetivo final requeridos por el sistema de navegación del robot.

VI-B. Validación experimental

Con el propósito de verificar el funcionamiento completo del sistema desarrollado, se realizaron veinte pruebas experimentales utilizando el micrófono integrado de la computadora. Durante estas pruebas se evaluó el funcionamiento conjunto del reconocimiento de voz, el procesamiento del lenguaje, la extracción automática de objetivos, el algoritmo de clasificación y la generación del archivo de parámetros requerido por el sistema de navegación.

Cada prueba consistió en pronunciar un comando de voz diferente, verificando posteriormente la intención predicha por el modelo, el objetivo inicial, el objetivo final y la información almacenada en el archivo de parámetros generado automáticamente por el nodo.

Las veinte pruebas realizadas fueron clasificadas correctamente por el sistema, obteniéndose una coincidencia total entre el comando pronunciado por el usuario y la información generada automáticamente por el nodo. En todos los casos se identificó correctamente la intención correspondiente al comando, así como los objetivos inicial y final cuando estos se encontraban presentes dentro de la instrucción.

Los resultados obtenidos permiten verificar el funcionamiento conjunto de todas las etapas desarrolladas durante el proyecto. La correcta integración entre el reconocimiento de voz mediante Vosk, el modelo de clasificación supervisada y el nodo de ROS permitió automatizar satisfactoriamente la generación de los parámetros requeridos por el sistema de navegación.

Como material complementario, dentro del directorio `/video` adjunto al repositorio del proyecto se incluye un vi-

Tabla II
VALIDACIÓN EXPERIMENTAL DEL SISTEMA DE CLASIFICACIÓN.

Comando	Clase	O_i	O_f	Res.
Ir al objetivo 2	GOAL	0	2	Correcta
Ir al objetivo 5	GOAL	0	5	Correcta
Del 1 al 4	PARTIAL	1	4	Correcta
Del 2 al 6	PARTIAL	2	6	Correcta
Trayectoria completa	COMPLETE	0	0	Correcta
Completar recorrido	COMPLETE	0	0	Correcta
Recorrido cíclico	CYCLIC	0	0	Correcta
Repetir trayectoria	CYCLIC	0	0	Correcta
Ir al objetivo 7	GOAL	0	7	Correcta
Del 3 al 8	PARTIAL	3	8	Correcta
Modo completo	COMPLETE	0	0	Correcta
Modo cíclico	CYCLIC	0	0	Correcta
Modo 3	COMPLETE	0	0	Correcta
Objetivo 9	GOAL	0	9	Correcta
Del 4 al 9	PARTIAL	4	9	Correcta
Ir al objetivo 1	GOAL	0	1	Correcta
Modo 2: 1 $\rightarrow$ 3	PARTIAL	1	3	Correcta
Recorrido completo	COMPLETE	0	0	Correcta
Trayectoria circular	CYCLIC	0	0	Correcta
Ir al objetivo 10	GOAL	0	10	Correcta

deo donde se muestra el funcionamiento completo del sistema desarrollado, desde la emisión del comando de voz hasta la generación automática del archivo de parámetros utilizado por el robot Husky A200.

VII. HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL UTILIZADAS

Durante el desarrollo del presente proyecto se emplearon herramientas de inteligencia artificial generativa como apoyo en diferentes etapas del trabajo.

En la etapa de desarrollo del software se utilizó *Claude* como herramienta de apoyo para la corrección del código fuente y el desarrollo de secciones específicas del programa implementado.

Por otra parte, se utilizó *ChatGPT* como herramienta de apoyo para diferentes actividades relacionadas con la elaboración del proyecto. En primer lugar, se empleó para la generación de la Figura 1, correspondiente a la representación de la estructura general del proyecto. Asimismo, se utilizó durante la revisión, corrección y mejora de la redacción del presente informe, con el propósito de aumentar la claridad, coherencia y organización del documento. También fue utilizado como apoyo para la generación de ejemplos de comandos de voz empleados durante la construcción de los conjuntos de entrenamiento y prueba.

VIII. CONCLUSIONES

El desarrollo del presente proyecto permitió implementar un sistema capaz de interpretar comandos de voz para automatizar la selección del modo de operación del robot Husky A200. La integración entre el reconocimiento de voz, el modelo de clasificación supervisada y el sistema de navegación permitió eliminar la necesidad de modificar manualmente los archivos de configuración, simplificando la interacción entre el usuario y el robot.

La construcción de un conjunto de datos compuesto por 607 comandos de entrenamiento y 153 comandos de prueba permitió evaluar objetivamente diferentes algoritmos de clasificación supervisada bajo las mismas condiciones experimentales. La incorporación de múltiples formas de expresar una misma instrucción favoreció la capacidad de generalización de los modelos frente a comandos no utilizados durante el entrenamiento.

La comparación entre los modelos Árbol de Decisión, Naive Bayes, K-Nearest Neighbors, SVM lineal y SVM con kernel RBF permitió determinar que este último constituye la alternativa más adecuada para el problema planteado, alcanzando un Accuracy de 0.9673 y un F1-score ponderado de 0.9674. Estos resultados evidencian que el empleo de técnicas de clasificación supervisada resulta apropiado para interpretar comandos emitidos en lenguaje natural dentro de aplicaciones robóticas.

Finalmente, las pruebas experimentales realizadas demostraron el correcto funcionamiento del sistema completo bajo condiciones reales de operación. Durante la validación, el nodo fue capaz de reconocer el comando pronunciado, identificar la intención correspondiente, extraer automáticamente los objetivos de navegación y generar el archivo de parámetros requerido por el robot, verificando la correcta integración entre todas las etapas desarrolladas.

REFERENCIAS

- [1] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [2] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” 2026. Recuperado de: <https://scikit-learn.org/>.
- [3] Scikit-learn Developers, “DecisionTreeClassifier,” 2026. Recuperado de: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>.
- [4] Scikit-learn Developers, “GaussianNB,” 2026. Recuperado de: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html.
- [5] Scikit-learn Developers, “KNeighborsClassifier,” 2026. Recuperado de: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>.
- [6] Scikit-learn Developers, “SVC,” 2026. Recuperado de: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>.
- [7] Scikit-learn Developers, “Classification metrics,” 2026. Recuperado de: https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics.
- [8] Open Robotics, “ROS Noetic Documentation,” 2026. Recuperado de: <https://wiki.ros.org/noetic>.
- [9] Alpha Cephei, “Vosk Speech Recognition Toolkit,” 2026. Recuperado de: <https://alphacephei.com/vosk/>.
- [10] NumPy Developers, “NumPy Documentation,” 2026. Recuperado de: <https://numpy.org/doc/>.
- [11] Pandas Developers, “Pandas Documentation,” 2026. Recuperado de: <https://pandas.pydata.org/docs/>.
- [12] Joblib Developers, “Joblib Documentation,” 2026. Recuperado de: <https://joblib.readthedocs.io/en/stable/>.